HORIZONX

HorizonX Technical Design Document

Frontend Architecture, Web3 Boundary & Implementation Notes

VERSION 1.0 | 4 November 2026

Document contents

1. 1. Document control
2. 2. Architecture summary
3. 3. Technology stack
4. 4. Runtime and component architecture
5. 5. Route and module topology
6. 6. Data architecture and persistence
7. 7. Web3 integration boundary
8. 8. Data provenance and UI truthfulness
9. 9. Security and privacy design
10. 10. Deployment and operations
11. 11. Quality and test design
12. 12. Technical risks and follow-up
13. 13. Visual design system

1. Document control

Field	Value
Document	Technical Design Document (TDD)
Project	HorizonX (repository/project label)
Current in-app brand	HorizonX references remain in application metadata and data
Version / date	1.0 / 4 November 2026
Architecture status	Frontend SPA implemented; backend and contract layers not present

2. Architecture summary

The uploaded application is a client-rendered React single-page application built with Vite and React Router. It is not the Next.js App Router architecture described by older documentation in the archive. The router is created with createBrowserRouter; page modules are loaded lazily and rendered inside a shared AppShell.

The application is frontend-only. There is no Express server, API service, MongoDB/Mongoose persistence, server-side authentication, indexer or smart-contract repository in the uploaded implementation.

3. Technology stack

Layer	Current implementation
Runtime / language	React 19.3.0, TypeScript 5.x, ES2022 target, strict TypeScript
Build / dev server	Vite 7.x, @vitejs/plugin-react, Tailwind CSS Vite plugin
Routing	React Router DOM 7.x, createBrowserRouter, route-level lazy imports
Styling / components	Tailwind CSS 4, shadcn component conventions, Radix UI primitives, project-owned UI wrappers
Wallet / EVM	Wagmi 3.x, Viem 2.x, injected / MetaMask / WalletConnect / Coinbase connectors
Network	Ethereum Sepolia only; Chain ID 11155111
Async query cache	TanStack Query 5.x; 5-second default stale time for configured queries
UI / motion / visualization	Motion, D3, Three.js, @react-three/fiber, Recharts is not declared in current package manifest
Notifications	Sonner
Tests	tsx-based TypeScript test files; package test script invokes 15 explicit files

4. Runtime and component architecture

- frontend/index.html supplies the browser document, metadata, font loading and #root mount point.
- frontend/src/main.tsx mounts React StrictMode and RouterProvider.
- frontend/src/routes.tsx defines 52 explicit page routes (home index plus 51 named routes) and a wildcard not-found route. Each page is imported through a lazy loader.
- AppShell provides the shared top navigation, page outlet, footer, scroll restoration, modal renderer, toast container and provider composition.
- Provider nesting: Web3Provider (WagmiProvider + QueryClientProvider) → WalletProvider → ModalProvider → application shell.
- Feature pages live under src/pages; reusable feature and design components live under src/components.
- Global design tokens and Tailwind v4 integration are defined in src/styles/globals.css; shared configuration is in src/config and src/data.

5. Route and module topology

Module	Representative routes
Home	/
Explore	/explore, /explore/tokens, /explore/projects, /explore/presales, /explore/launches, /explore/airdrops, /explore/staking
Tokens	/tokens, /tokens/explore, /tokens/my, /tokens/create, /tokens/watchlist, /tokens/:id, /tokens/:id/contract, /tokens/:id/activity
Launchpad	/launchpad, /launchpad/presales, /launchpad/presales/create, /launchpad/presales/:id, /launchpad/launches, /launchpad/launches/create, /launchpad/launches/:id, /launchpad/my
Airdrops	/airdrops, /airdrops/campaigns, /airdrops/campaigns/create, /airdrops/campaigns/:id, /airdrops/claim, /airdrops/my
Staking	/staking, /staking/pools, /staking/pools/create, /staking/pools/:id, /staking/my
Exchange	/exchange, /exchange/markets, /exchange/tokens/:id, /exchange/watchlist
Projects	/projects, /projects/my, /projects/:id
Partners	/partners, /partners/referrals
Account	/account, /account/edit, /account/preferences, /account/security, /account/verification, /account/wallets
Docs / Help	/docs, /docs/:slug

6. Data architecture and persistence

Canonical catalog data is exported through src/data/index.ts from src/data/catalog. Current seed record counts in the inspected source are 9 tokens, 10 projects, 8 presales, 8 fair launches, 8 airdrop campaigns and 8 staking pools (51 records total). These are illustrative fixtures, not a live database or indexed chain dataset.

Data class	Storage/source	Persistence / trust boundary
Catalog	src/data/catalog/*.ts	Bundled with frontend; static seed records, not chain-verified.
Simulated metrics	Token/market demo fixtures and derived selectors	Illustrative values; never a live market feed.
Token creation receipts	sessionStorage via prominers_session_tokens	Current browser session; no contract deployment.
Launch drafts/records	sessionStorage via pm_session_launches_v1	Session-created simulation records; no fundraising transaction.
Airdrop records	sessionStorage via pm_session_airdrops_v1	Session-created simulation records; no on-chain distribution.
Watchlists / demo wallet	Browser storage keys in feature contexts	Device/browser-local state; not synchronized across users or devices.
Wallet account/balance	Wagmi connector and Sepolia RPC	Real wallet provider state and Sepolia native ETH balance; no platform account authentication.
Backend/database	Not present	No durable server-side records, cross-device sync or server authorization.

7. Web3 integration boundary

- wagmi configuration registers viem's Sepolia chain only and configures injected, MetaMask, WalletConnect and Coinbase Wallet connectors.
- WalletContext exposes connection, address, connector, network state, demo mode and Sepolia balance. It detects unsupported chains and offers a Sepolia switch path.

- Demo wallet mode is a local simulation. It uses a fixed illustrative address and balance and persists a demo-active flag in localStorage. It is not a cryptographic wallet session.
- Real wallet connection is not equivalent to user authentication or authorization on a backend.
- Token catalog entries may mention networks such as Arbitrum, Base and Polygon, but the active wallet configuration does not support those networks in this baseline.
- Token-specific contract reads, token deployment, presale/fair-launch contracts, airdrop claims, staking contracts, swaps and a transaction indexer are not implemented.

8. Data provenance and UI truthfulness

Provenance	Meaning	Allowed claim
catalog	Static bundled record	Catalog information; not independently chain-verified.
simulated	Generated illustrative value	Demo / simulated only; no live market or chain implication.
session	Created in current browser session	Session record; not a persistent platform record or deployed asset.
onchain	Reserved type for verified chain reads	Only use after implementing and verifying the corresponding chain read. Token-specific on-chain data is absent in this baseline.

9. Security and privacy design

- All VITE_ environment variables are compiled into browser-accessible code; they are configuration, not secrets.
- Never put private keys, seed phrases, signing credentials, database passwords or provider secrets in frontend environment variables.
- The WalletConnect project ID is a public client identifier, not a server-side secret. Configure a project ID owned by the deploying organization rather than relying on a shared fallback.
- Do not treat a connected wallet, catalog verification label or demo session as KYC, identity verification, project approval or security certification.
- Browser storage is user-controlled and can be cleared or modified. It is not suitable for trusted authorization, financial records or authoritative business state.
- Production privileged actions require server-side authorization, audit logging, replay/idempotency controls and explicit contract-level access control; none is present here.

10. Deployment and operations

- Run the Vite production build from the frontend directory; output is frontend/dist.
- Deploy the static dist output to a static hosting provider.
- Configure an SPA rewrite/fallback so direct requests to nested paths return index.html. frontend/public/_redirects contains a Netlify-style fallback rule.
- Set VITE_APP_URL, VITE_APP_NAME, VITE_DEMO_MODE, VITE_DEFAULT_CHAIN_ID, VITE_WALLETCONNECT_PROJECT_ID and optional VITE_SEPOLIA_RPC_URL / VITE_EXPLORER_URL before building.
- Because Vite values are baked into the client bundle at build time, rebuild after changing public environment configuration.
- No CI workflow, backend deployment manifest, database migration or production observability configuration is evidenced in the supplied archive.

11. Quality and test design

Check	Repository evidence	Documentation status
Build	npm run build → vite build	Not executed during this documentation pass
Typecheck	npm run typecheck → tsc --noEmit	Not executed
Lint	npm run lint → eslint .	Not executed
Tests	28 *.test.ts files exist under src/tests	Not executed
Configured test command	npm run test explicitly chains 15 test files	Does not invoke all 28 discovered test files; test script coverage gap recorded

The test files cover wallet state, token directories and lifecycle, data provenance, presale/fair-launch flows, airdrop creation/Merkle helpers, staking directories and launch session workflows. File presence is not proof that tests pass. No runtime wallet-provider, RPC outage, contract transaction, production deployment or browser end-to-end test result is established by this inspection.

12. Technical risks and follow-up

Risk / gap	Impact	Required next action
No backend or database	No durable records, secure roles, cross-device state or trusted APIs.	Approve backend architecture and data ownership before expanding account workflows.
No smart contracts	Creation and campaign actions are not real protocol operations.	Specify, implement, test and independently audit each contract family before production.
Market data is illustrative	Users could misread mock figures as actual market conditions.	Integrate named provider, source attribution, freshness and stale/unavailable states.
Network mismatch	Catalog records mention networks beyond active Sepolia support.	Normalize supported network metadata and block unsupported actions clearly.
Incomplete test command	13 of 28 test files are not invoked by npm test.	Use a glob-based runner or explicitly chain every intended test and verify results.
No production controls	No monitoring, backup, incident response or release pipeline is evidenced.	Define operational readiness and release gates.

13. Visual design system

The current frontend stylesheet defines a HorizonX dark graphite theme. This differs from the older docs/DESIGN_SYSTEM.md record, which specifies a warm-black/orange HorizonX palette. The current source code is authoritative for the running interface.

Token / purpose	Current source value	Usage
Page background	`#0B0806`	Global dark canvas
Primary surface	`#121518`	Cards and primary surfaces
Elevated surface	`#191D21`	Raised panels and controls
Primary text	`#E5E9F4`	Main readable text
Muted text	`#8793AD`	Secondary labels and supporting text
Primary accent / focus	`#A99AFF`	Primary actions, focus and brand highlight
Secondary brand accent	`#6FDBE9`	Brand gradient endpoint
Borders	`#2A3037` / `#3A424B`	Subtle and strong separators
Typography	Outfit, Fraunces, IBM Plex Mono	Sans, display/heading and monospace roles

The code should be reconciled with the historical design-system document before that document is treated as a current design authority.